

PYTHON AUTOMATION – MODULE-WISE LAB

Submitted by

Name: Ananya Saha

Enrollment No: 12023002001198

Stream: CSE

Institute: Institute Of
Engineering & Management,
Kolkata

INDEX

Sl. No.	Assignment	Page No.
1	The Multi-Locator Challenge	3
2	Synchronization and Explicit Waits	4
3	Dynamic Dropdowns and Checkboxes	5
4	JavaScript Alerts and Confirms	6
5	HTML Web Table Extractor	7
7	Page Object Model Restructure	8
8	Data-Driven Automation	9
9	PyTest Integration and HTML Reporting	10
10	Unittest – First Test Case	11
11	PyTest Fixtures and conftest.py	12
12	PyTest Parameterization	12
13	Selenium Python and Behave BDD	13
14	Test Data Driven Automation in Behave	13
15	Selenium Page Object Model in Python Behave	14
16	REST API – GET Request	14
17	REST API – POST Request	14
18	Robot Framework – Basic Syntax and Keywords	15
19	Robot Framework – Variables and Data Driven Testing	16
20	Robot Framework – Custom Keywords	17
21	Robot Framework – Assertions and Verification	18
22	Robot Framework – Setup and Teardown	19
23	Robot Framework – Tags and Test Execution	20
24	Robot Framework – Reports and Logs	20

MODULE 1 – AUTOMATION WITH SELENIUM

EXPERIMENT 1: THE MULTI-LOCATOR CHALLENGE

Aim

To automate a login page using different Selenium locator strategies.

Requirements

- Python
- Selenium
- Google Chrome

Procedure

1. Open the SauceDemo website.
2. Locate the username field using ID.
3. Locate the password field using NAME.
4. Locate the login button using XPATH.
5. Enter the login details.
6. Click Login.
7. Verify the resulting URL.

Program

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 driver = webdriver.Chrome()
4 driver.get("https://www.saucedemo.com/")
5 driver.find_element(By.ID, "user-name").send_keys("standard_user")
6 driver.find_element(By.NAME, "password").send_keys("secret_sauce")
7 driver.find_element(
8     By.XPATH, "//input[@value='Login']"
9 ).click()
10 assert "/inventory.html" in driver.current_url
11 print("Login successful")
12 driver.quit()
13
```

Output

Login successful

Result

The login functionality was successfully automated using different Selenium locators.

EXPERIMENT 2: SYNCHRONIZATION AND EXPLICIT WAITS

Aim

To handle dynamic web elements using explicit waits.

Procedure

1. Open the dynamic loading page.
2. Click the Start button.
3. Wait for the element to become visible.
4. Display the text.

Program

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 driver = webdriver.Chrome()
6 driver.get(
7     "https://the-internet.herokuapp.com/dynamic_loading/1"
8 )
9 driver.find_element(
10     By.CSS_SELECTOR,
11     "#start button"
12 ).click()
13 element = WebDriverWait(driver, 10).until(
14     EC.visibility_of_element_located(
15         (By.ID, "finish")
16     )
17 )
18 print(element.text)
19 driver.quit()
```

Result

The dynamic element was successfully handled using WebDriverWait.

EXPERIMENT 3: DYNAMIC DROPDOWNS AND CHECKBOXES

Aim

To automate checkboxes and dropdown elements using Selenium.

Procedure

1. Open the required web page.
2. Locate the checkbox.
3. Check whether it is already selected.
4. Select it if required.
5. Enter the required value in the dropdown/search field.
6. Select the required option.

Program

```
1 checkbox = driver.find_element(  
2     By.ID,  
3     "checkbox_id"  
4 )  
5  
6 if not checkbox.is_selected():  
7     checkbox.click()  
8  
9 assert checkbox.is_selected()  
10  
11 print("Checkbox selected")  
12
```

Output

Checkbox selected

Result

The checkbox was selected and its state was successfully verified.

EXPERIMENT 4: JAVASCRIPT ALERTS AND CONFIRMS

Aim

To handle JavaScript Alert, Confirm and Prompt boxes.

Procedure

- Open the JavaScript alerts page.
- Accept the alert.
- Dismiss the confirmation box.
- Enter text in the prompt.
- Accept the prompt.

Program

```
1 from selenium.webdriver.common.by import By
2 driver.find_element(
3     By.XPATH,
4     "//button[text()='Click for JS Alert']"
5 ).click()
6 alert = driver.switch_to.alert
7 print(alert.text)
8 alert.accept()
9 driver.find_element(
10     By.XPATH,
11     "//button[text()='Click for JS Confirm']"
12 ).click()
13 alert = driver.switch_to.alert
14 alert.dismiss()
15 driver.find_element(
16     By.XPATH,
17     "//button[text()='Click for JS Prompt']"
18 ).click()
19 alert = driver.switch_to.alert
20 alert.send_keys("Test Input")
21 alert.accept()
22
```

Result

Alert, Confirm and Prompt boxes were successfully handled.

EXPERIMENT 5: HTML WEB TABLE EXTRACTOR

Aim

To extract data from an HTML web table.

Procedure

1. Locate the table.
2. Find all table rows.
3. Read the cells.
4. Search for the required row.
5. Print the required column value.

Program

```
1 rows = driver.find_elements(  
2     By.CSS_SELECTOR,  
3     "table tbody tr"  
4 )  
5 for row in rows:  
6     cells = row.find_elements(  
7         By.TAG_NAME,  
8         "td"  
9     )  
10    values = [cell.text for cell in cells]  
11    if values and values[0] == "John":  
12        print("Matching Row:", values)  
13        print("Required Value:", values[2])  
14        break  
15
```

Result

The required row and column value were successfully extracted from the table.

EXPERIMENT 6: WINDOWS, TABS AND IFRAMES

Aim

To handle multiple browser windows, tabs and iframes.

Program

```
1 main_window = driver.current_window_handle  
2 iframe = driver.find_element(  
3     By.TAG_NAME,  
4     "iframe"  
5 )  
6 driver.switch_to.frame(iframe)  
7 print("Inside iframe")  
8 driver.switch_to.default_content()  
9 handles = driver.window_handles  
10 for handle in handles:  
11     if handle != main_window:  
12         driver.switch_to.window(handle)  
13         print(driver.title)  
14         driver.close()  
15         break  
16 driver.switch_to.window(main_window)  
17
```

Result

The iframe and multiple browser windows were successfully handled.

EXPERIMENT 7: PAGE OBJECT MODEL RESTRUCTURE

Aim

To organize Selenium automation using Page Object Model.

Project Structure

Assignment

```
|
|
|  └─ pages
|
|    └─ base_page.py
|
|    └─ login_page.py
|
|    └─ search_page.py
|
|
|  └─ tests
|
|    └─ test_login.py
|
|    └─ test_search.py
|
|
|  └─ utils
|
|
└─ config
```

Example

```
1  class LoginPage:
2      def __init__(self, driver):
3          self.driver = driver
4      def click_login(self):
5          self.driver.find_element(
6              By.XPATH,
7              "//input[@value='Login']"
8          ).click()
9
```

Result

The page-related operations were separated from the test cases using Page Object Model.

EXPERIMENT 8: DATA-DRIVEN AUTOMATION

Aim

To execute test cases using external test data.

Example CSV

username,password,expected

testuser1@example.com,testpass1,Fail

invaliduser@example.com,wrongpass,Fail

Program

```
1 import csv
2 with open(
3     "testdata/logindata.csv",
4     newline=""
5 ) as file:
6     data = list(
7         csv.DictReader(file)
8     )
9 for row in data:
10     print(row["username"])
11     print(row["password"])
12     print(row["expected"])
13
```

Result

The test data was successfully read from the external CSV file.

EXPERIMENT 9: PYTEST INTEGRATION AND HTML REPORTING

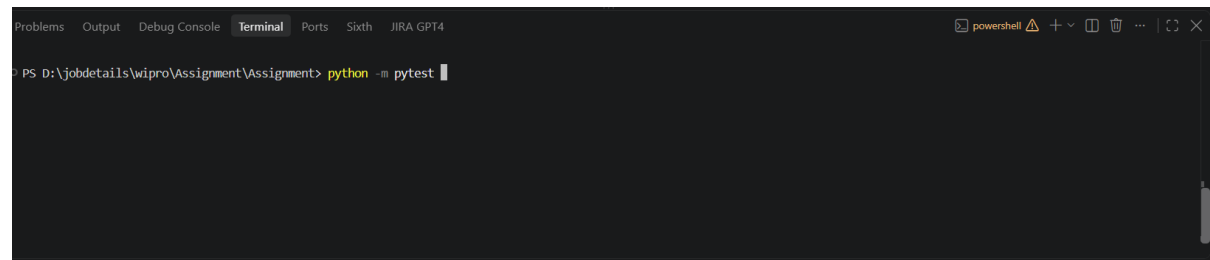
Aim

To execute Selenium test cases using PyTest and generate an HTML report.

Program

```
1 import pytest
2 from selenium import webdriver
3 @pytest.fixture()
4 def setup():
5     driver = webdriver.Chrome()
6     driver.maximize_window()
7     yield driver
8     driver.quit()
9 def test_home_page(setup):
10     setup.get(
11         "https://tutorialsninja.com/demo/"
12     )
13     assert "Your Store" in setup.title
14
```

Execution Command



```
PS D:\jobdetails\wipro\Assignment\Assignment> python -m pytest
```

Output

6 passed

Result

The Selenium tests were successfully executed using PyTest and the HTML report was generated.

MODULE 2 – UNIT TEST FRAMEWORKS

EXPERIMENT 10: UNITTEST – FIRST TEST CASE

Aim

To create and execute a test case using Python Unittest.

Program

```
1 import unittest
2 class TestExample(unittest.TestCase):
3     def setUp(self):
4         print("Setup")
5     def test_addition(self):
6         self.assertEqual(
7             2 + 2,
8             4
9         )
10    def tearDown(self):
11        print("TearDown")
12 if __name__ == "__main__":
13     unittest.main()
14
```

Output

OK

Result

The test case was successfully executed using Unittest.

EXPERIMENT 11: PYTEST FIXTURES AND CONFTEST.PY

Aim

To create reusable browser setup and teardown using PyTest fixtures.

conftest.py

```
1 import pytest
2 from selenium import webdriver
3
4 @pytest.fixture()
5
6 def setup():
7
8     driver = webdriver.Chrome()
9
10    driver.maximize_window()
11
12    yield driver
13
14    driver.quit()
```

Test

```
1 def test_home_page(setup):
2
3     setup.get(
4         "https://tutorialsninja.com/demo/"
5     )
6
7     assert "Your Store" in setup.title
```

Result

The browser was automatically opened before the test and closed after execution

EXPERIMENT 12: PYTEST PARAMETERIZATION

Aim

To execute the same test using different sets of test data.

Program

```
1 import pytest
2
3 @pytest.mark.parametrize(
4     "product",
5     [
6         "MacBook",
7         "iPhone",
8         "InvalidProductxyz"
9     ]
10 )
11
12 def test_search(product):
13
14     print(
15         "Searching:",
16         product
17     )
```

Result

The same test was executed with multiple product values.

MODULE 3 – PYTHON BDD RESTFUL AUTOMATIONS

EXPERIMENT 13: SELENIUM PYTHON AND BEHAVE BDD

Aim

To create a BDD automation scenario using Python, Selenium and Behave.

Feature File

Feature: Login Test

Scenario: Open login page

Given I open the application

When I open the login page

Then the login page should be displayed

Execution

behave

Result

The BDD scenario was executed using the Given-When-Then format.

EXPERIMENT 14: TEST DATA DRIVEN AUTOMATION IN BEHAVE

Aim

To execute a BDD scenario using multiple test data values.

Program

Feature: Product Search

Scenario Outline: Search product

Given I open the application

When I search for "<product>"

Then the search result should be displayed

Examples:

product
MacBook
iPhone

Result

The same scenario was executed using multiple product values.

EXPERIMENT 15: SELENIUM PAGE OBJECT MODEL IN PYTHON BEHAVE

Aim

To integrate Page Object Model with Selenium and Behave.

Example

```
class LoginPage:
    def __init__(self, driver):
        self.driver = driver

    def open_login(self):
        # Selenium operations
        pass
```

Result

The Selenium operations were organized into page classes and used by Behave step definitions.

EXPERIMENT 16: REST API – GET REQUEST

Aim

To send a GET request using Python Requests.

Program

```
1 import requests
2 response = requests.get(
3     "https://jsonplaceholder.typicode.com/posts/1"
4 )
5 print(
6     "Status:",
7     response.status_code
8 )
9 print(
10     response.json()
11 )
12 assert response.status_code == 200
13
```

Result

The GET request was successfully executed and the response status was verified.

EXPERIMENT 17: REST API – POST REQUEST

Aim

To send a POST request with JSON data.

Program

```
1 import requests
2 payload = {
3     "title": "Automation Test",
4     "body": "Python Automation",
5     "userId": 1
6 }
7 response = requests.post(
8     "https://jsonplaceholder.typicode.com/posts",
9     json=payload
10 )
11 print(response.status_code)
12 print(response.json())
```

Result

The POST request was successfully sent with the required JSON data.

MODULE 4 – ROBOT FRAMEWORK

EXPERIMENT 18: BASIC SYNTAX AND KEYWORDS

Aim

To create a basic Robot Framework test case.

Program

*** Settings ***

Library SeleniumLibrary

*** Test Cases ***

Open Website

Open Browser

... <https://tutorialsninja.com/demo/>

... chrome

Title Should Contain

... Your Store

[Teardown]

... Close All Browsers

Result

The website was opened and the title was verified.

EXPERIMENT 19: VARIABLES AND DATA-DRIVEN TESTING

Aim

To use variables in Robot Framework.

Program

*** Variables ***

\${URL} https://tutorialsninja.com/demo/

\${BROWSER} chrome

*** Test Cases ***

Open Store

Open Browser

... \${URL}

... \${BROWSER}

Title Should Contain

... Your Store

[Teardown]

... Close All Browsers

Result

Variables were successfully used in the Robot Framework test.

EXPERIMENT 20: CUSTOM KEYWORDS

Aim

To create reusable custom keywords.

Program

*** Keywords ***

Open Store

Open Browser

... <https://tutorialsninja.com/demo/>

... chrome

Title Should Contain

... Your Store

*** Test Cases ***

Verify Store

Open Store

[Teardown]

... Close All Browsers

Result

A custom keyword was created and reused successfully.

EXPERIMENT 21: ASSERTIONS AND VERIFICATION

Aim

To verify actual results against expected results.

Program

*** Test Cases ***

Verify Title

Open Browser

... <https://tutorialsninja.com/demo/>

... chrome

Title Should Contain

... Your Store

[Teardown]

... Close All Browsers

Result

The expected page title was successfully verified.

EXPERIMENT 22: TEST SETUP AND TEARDOWN

Aim

To use common setup and teardown in Robot Framework.

Program

*** Settings ***

Library SeleniumLibrary

Test Setup

Open Browser

... <https://tutorialsninja.com/demo/>

... chrome

Test Teardown

Close All Browsers

*** Test Cases ***

Check Website

Title Should Contain

... Your Store

Result

The browser was automatically opened before the test and closed after the test.

EXPERIMENT 23: TAGS AND TEST EXECUTION

Aim

To execute selected Robot Framework test cases using tags.

Program

*** Test Cases ***

Smoke Test

[Tags] smoke

Log Smoke test

Regression Test

[Tags] regression

Log Regression test

Command

robot --include smoke .

Result

Only the test cases having the selected tag were executed.

EXPERIMENT 24: ROBOT FRAMEWORK REPORTS AND LOGS

Aim

To generate reports and logs after Robot Framework execution.

Command

robot tests/

Generated Files

output.xml

log.html

report.html

Result

Robot Framework generated the execution output, log and report files successfully.

